Ministerul Educaţiei şi Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică şi Microelectronică

Laboratory work 5:

Study and Empirical Analysis of Prim and Kruskal Greedy Algorithms

Elaborated:
st. gr. FAF-241 Pleşu Dinu

Verified:
asist. univ. Fiştic Cristofor

Chişinău - 2026

TABLE OF CONTENTS

LIST OF ABBREVIATIONS AND DEFINITIONS **3**

INTRODUCTION **4**

1 ALGORITHM ANALYSIS **5**

2 IMPLEMENTATION **6**

2.1 Prim 6

2.2 Kruskal 7

CONCLUSION **10**

REFERENCES **11**

LIST OF ABBREVIATIONS AND DEFINITIONS

- greedy algorithm - method that builds a solution by choosing the best local option at each step;
- KB - kilobytes;
- Kruskal - greedy algorithm that constructs a minimum spanning tree by selecting edges in increasing order;
- ms - milliseconds;
- MST - Minimum Spanning Tree;
- Prim - greedy algorithm that grows a minimum spanning tree from an initial vertex.

INTRODUCTION

Prim and Kruskal are classical greedy algorithms for constructing a minimum spanning tree in a connected weighted graph. Both aim to connect all vertices with minimum total weight, but they do so using different strategies.

Prim grows one tree progressively, while Kruskal builds the solution by choosing edges in increasing order and avoiding cycles. Because both algorithms solve the same problem, they are appropriate for empirical comparison on graphs with different sizes and densities.

1 ALGORITHM ANALYSIS

Objective

Study and analyze Prim and Kruskal on connected weighted undirected graphs with increasing size and different density.

Tasks

1. study the greedy algorithm design technique;
2. implement Prim and Kruskal in a programming language;
3. perform empirical analysis of the two algorithms;
4. increase the number of nodes in the graph and analyze the influence on the algorithms;
5. make a graphical presentation of the obtained data;
6. formulate conclusions.

Theoretical Notes

Greedy algorithms build a solution step by step by selecting the best local option available at the current moment. For the minimum spanning tree problem, Prim expands a partial tree by adding the cheapest possible outgoing edge, while Kruskal sorts the edges and adds them one by one as long as no cycle is formed [1, 2].

The practical performance of these algorithms depends strongly on the internal data structures. Heap-based edge selection improves Prim, while an efficient disjoint-set structure improves Kruskal [3].

Comparison Metric

The selected metrics were:

– execution time in milliseconds;
– peak memory usage in kilobytes;
– total weight of the minimum spanning tree;
– number of edges and graph density.

Input Format

The benchmark used connected weighted undirected graphs with random edge weights from 1 to 20. Three graph categories were generated:

– sparse graphs;
– medium-density graphs;
– dense graphs.

The tested graph sizes were 100, 200, 350, 500, and 700 vertices.

2 IMPLEMENTATION

2.1 Prim

The project contains a classic Prim implementation and an optimized Prim implementation. The optimized variant uses a priority queue to select the next cheapest edge more efficiently [4].

Algorithm Description

The method follows the idea shown in the next pseudocode.

```
Prim(graph, start):
    mark start as part of the tree
    push all outgoing edges of start in priority queue
    while the queue is not empty:
        extract the cheapest edge
        if it reaches an unvisited vertex:
            add that edge to the tree
            add outgoing edges of the new vertex to the queue
    return minimum spanning tree
```

Implementation

The classic version keeps a simpler selection process, while the optimized version uses a more suitable heap-based approach. This improves execution time, but increases auxiliary memory usage.

Results

After executing the Prim implementations on all graph sizes and graph types, the representative values for the largest dense graph are shown in Table 2.1.

Table 2.1 – MST benchmark on dense graphs at 700 nodes

Algorithm	Time (ms)	Memory (KB)	MST weight	Edges
Kruskal_Classic	589.288	1582.46	699	122760.0
Kruskal_Optimized	545.990	1440.25	699	122760.0
Prim_Classic	1330.551	3879.47	699	122760.0
Prim_Optimized	435.591	7302.00	699	122760.0

In Table 2.1 it can be observed that the optimized Prim variant achieved the best running time among all implementations on the dense graph, with 435.592 ms. At the same time, it required the largest memory usage, which shows the trade-off introduced by the optimization. The overall MST performance is represented in Figure 2.1.

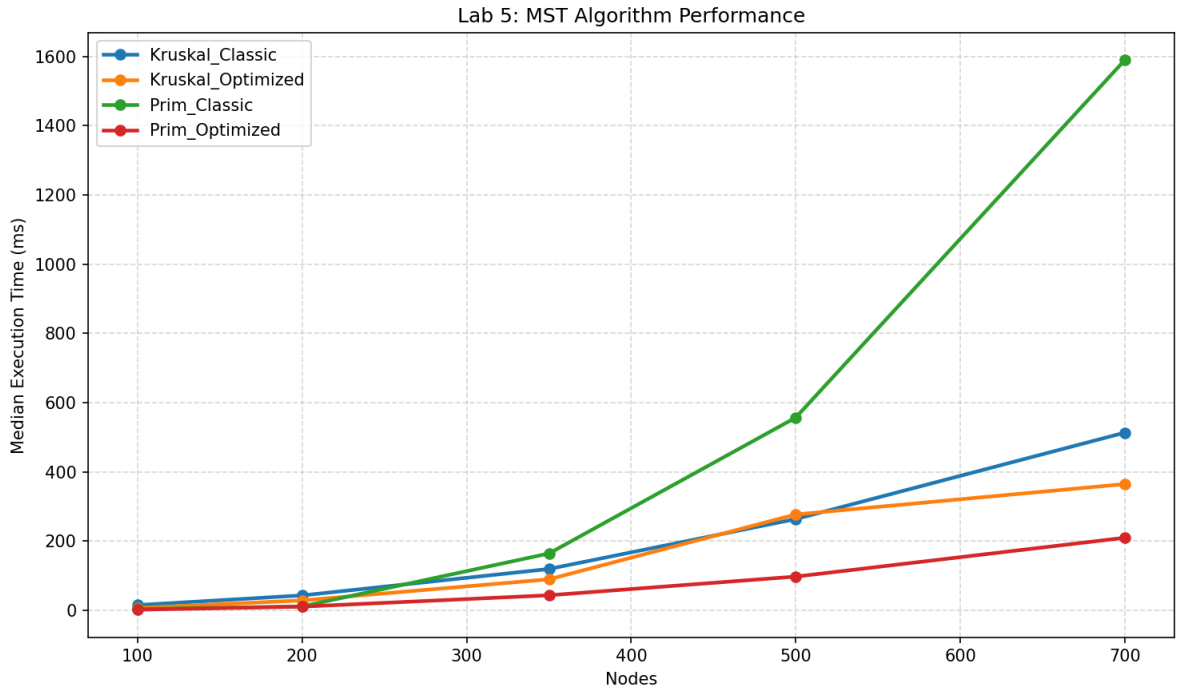

Figure 2.1 – Overall MST performance

Figure 2.1 shows the strong increase in execution time as the number of nodes grows. The growth is especially visible for the classic Prim variant.

Conclusion

Prim is effective for minimum spanning tree construction, but its practical performance depends heavily on the data structure used for edge selection. The optimized version is clearly preferable for large graphs when execution time is the main priority.

2.2 Kruskal

The project also contains a classic and an optimized Kruskal implementation. The optimized variant uses a disjoint-set structure to detect cycles efficiently.

Algorithm Description

The Kruskal method follows the idea shown in the next pseudocode.

```
Kruskal(graph):
    sort all edges increasingly by weight
    make one component for each vertex
    for each edge in sorted order:
        if the two endpoints are in different components:
            add edge to the tree
            merge the two components
    return minimum spanning tree
```

Implementation

The optimized version uses faster component management through union-find operations. This reduces both running time and memory compared with the simpler component-merging approach of the classic version.

Results

After the execution of Kruskal on sparse, medium, and dense graphs, the representative values for the largest sparse graph are shown in Table 2.2.

Table 2.2 – MST benchmark on sparse graphs at 700 nodes

Algorithm	Time (ms)	Memory (KB)	MST weight	Edges
Kruskal_Classic	362.709	458.16	715	25219.0
Kruskal_Optimized	227.098	315.95	715	25219.0
Prim_Classic	1318.228	3879.47	715	25219.0
Prim_Optimized	126.301	1421.54	715	25219.0

In Table 2.2 it can be observed that the optimized Kruskal variant improved both execution time and memory usage compared with the classic implementation. On the sparse graph with 700 nodes, the optimized version required 227.098 ms, while the classic version required 362.709 ms. The influence of graph type is represented in Figure 2.2.

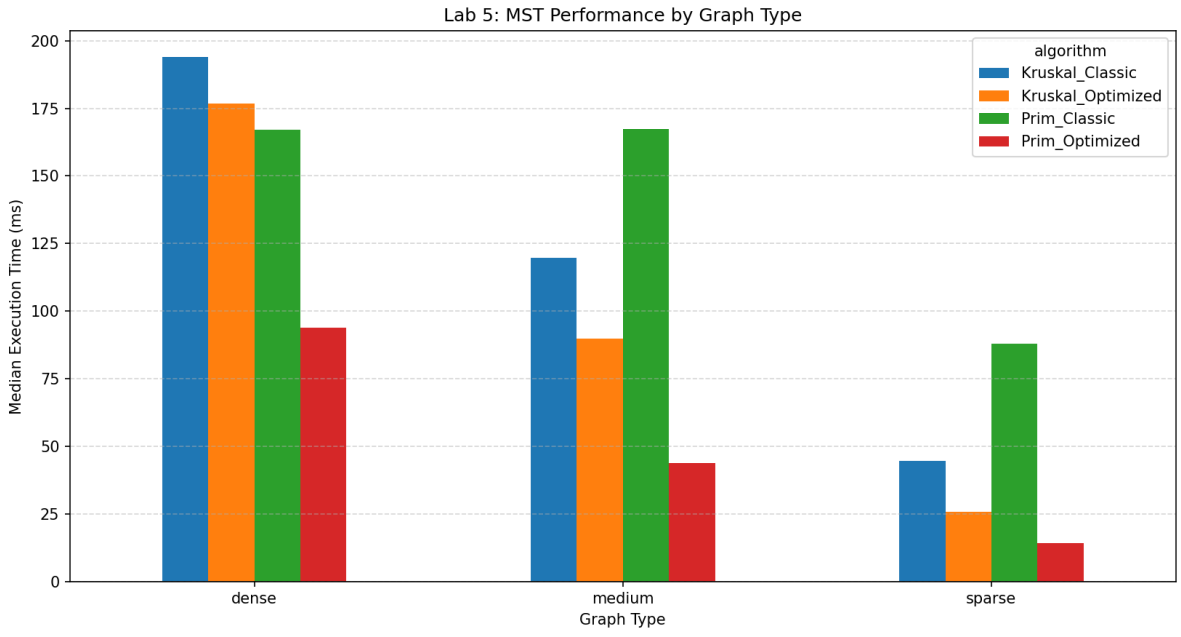

Figure 2.2 – MST performance by graph type

Figure 2.2 shows how graph density influences all tested algorithms. Dense graphs systematically lead to higher running times because they contain many more candidate edges to inspect.

Conclusion

Kruskal remains a strong greedy solution for MST construction, especially when combined with an efficient disjoint-set structure. The optimized version is preferable for larger graphs because it improves both speed and memory behavior.

CONCLUSION

The experiments confirm that graph size and density strongly influence greedy minimum-spanning-tree algorithms. Dense graphs increase the amount of work for both Prim and Kruskal, while optimized data structures provide important practical gains.

Among the tested variants, the optimized Prim implementation achieved the best overall running time, while the optimized Kruskal implementation provided a very good balance between speed and memory usage. The results therefore support the main idea of greedy algorithm analysis: the design principle is important, but the practical efficiency depends decisively on the implementation details.

REFERENCES

REFERENCES

[1] GeeksforGeeks. *Greedy Algorithms* [online]. [cited 07.05.2026]. Available: geeksforgeeks.org/greedy-algorithms.

[2] TutorialsPoint. *Greedy Algorithms* [online]. [cited 07.05.2026]. Available: tutorialspoint.com/data_structures_algorithms/greedy_algorithms.htm.

[3] Scaler. *Prim's and Kruskal's Algorithm* [online]. [cited 07.05.2026]. Available: scaler.com/topics/prims-and-kruskal-algorithm.

[4] GitHub Repository. *Lab 5 source folder* [online]. [cited 07.05.2026]. Available: github.com/Prince-of-Nothing/aa-course-repo/tree/main/Lab5.